JavaScript para Desarrollo Front End

Programación Full Stack - 3ro Bilingüe TT

Transición Java → JavaScript

PokeAPI + DOM

Fetch + LocalStorage

Prof. Carlos Romero

2026

Contenidos del Curso

JavaScript para Desarrollo Front End - 6 semanas de aprendizaje



Unidad 0 2 clases

Transición Java → JS

Variables, scope, arrow functions, tipado dinámico



Unidad 1 ⭐ CLAVE

DOM

Selectores, innerHTML, createElement, classList



Unidad 2 2 clases

Eventos

addEventListener, click, submit, preventDefault



Unidad 3 ⭐ CRÍTICO

Fetch y APIs

Promesas, async/await, PokeAPI, verbos HTTP



Unidad 4 2 clases

Formularios

Inputs, validación, mensajes de error



Unidad 5 2 clases

Objetos, Clases y localStorage

Objetos literales, clases JS, sessionStorage



Unidad 6 Proyecto Final

Integrador Pokémon

Galería completa + CRUD + Integración DOM + Eventos + Fetch

Unidad 0 2 clases

Transición Java → JavaScript

De Java a JavaScript: Cambiando de paradigma

Java

Lo que ya conocés

- **Tipado estático**
  int, String, boolean...
- **Orientado a objetos**
  Clases y herencia
- **Compilado**
  JVM + bytecode
- **Escritorio / Backend**
  Swing, Spring

Cambio de paradigma

JavaScript

Lo que vamos a aprender

- **Tipado dinámico**
  let, const, var
- **Multiparadigma**
  Objetos, funciones, eventos
- **Interpretado**
  Navegador / Node.js
- **Frontend / Backend**
  DOM + Fetch

Consejo clave

Tratá a JavaScript como un lenguaje nuevo. No intentes "traducir" Java a JS, aprendé su forma de pensar.

Variables y Scope en JavaScript

Declaraciones en JavaScript

```
var nombre = "Carlos"; // Function scope
let edad = 25; // Block scope
const PI = 3.14159; // Block scope + inmutable
```

¡Cuidado con var!
Tiene function scope y hoisting, puede causar bugs

Java vs JavaScript: Tipado

Java (Estático)

```
int x = 10;
String s = "Hola";
boolean ok = true;
```

JavaScript (Dinámico)

```
let x = 10;
let s = "Hola";
let ok = true;
```

JS infiere el tipo automáticamente

Scope en JavaScript

Global Scope

```
let pokemon = "Pikachu"; // Accesible en todo el archivo
```

Function Scope

```
function capturar() {
  let pokemon = "Charizard"; // Solo dentro de la función
}
```

Block Scope (let/const)

```
if (true) {
  let pokemon = "Bulbasaur"; // Solo dentro del bloque
}
console.log(pokemon); // ✕ Error!
```

Mejor práctica
Siempre usá **let** o **const** para evitar problemas de scope

Template Literals y Funciones

Tт Template Literals

✖ Concatenación antigua

```
let nombre = "Pikachu";
let tipo = "Eléctrico";
let msg = nombre + " es " +
tipo;
```

✔ Template Literal

```
let nombre = "Pikachu";
let tipo = "Eléctrico";
let msg = `${nombre} es
${tipo}`;
```

Características

- ✓ Usan backticks (`) en vez de comillas
- ✓ Expresiones con ${} dentro del string
- ✓ Strings multilínea sin \n

Tip
Template literals hacen el código más legible y fácil de mantener

Σ Funciones: Tradicional → Arrow

PASO 1 Función Tradicional

```
function saludar(nombre) {
  return `Hola ${nombre}`;
}
```

PASO 2 Arrow Function Completa

```
const saludar = (nombre) ⇒ {
  return `Hola ${nombre}`;
}
```

PASO 3 Arrow Function Simplificada

```
const saludar = nombre ⇒ `Hola ${nombre}`;
```

Sobre this
Veremos el comportamiento de this en arrow functions en la Unidad 5 (Clases)

Objetos y Arrays en JavaScript

{} Objetos Literales

Java (Clases)

```
class Pokemon {
  String nombre;
  String tipo;
  Pokemon(String n, String t) {
    nombre = n;
    tipo = t;
  }
}
```

JavaScript (Objeto Literal)

```
const pokemon = {
  nombre: "Pikachu",
  tipo: "Eléctrico",
  atacar() {
    return "¡Impactrueno!";
  }
}
```

Acceso a Propiedades

```
pokemon.nombre // Notación de punto
pokemon["tipo"] // Notación de corchetes
```

Usá corchetes cuando el nombre de la propiedad sea dinámico

Arrays y Métodos

```
const pokemones = ["Pikachu", "Charizard", "Bulbasaur"];
```

.map() – Transformar

```
pokemones.map(p ⇒ p.toUpperCase())
// ["PIKACHU", "CHARIZARD", "BULBASAUR"]
```

.filter() – Filtrar

```
pokemones.filter(p ⇒ p.length > 7)
// ["CHARIZARD", "BULBASAUR"]
```

.find() – Buscar

```
pokemones.find(p ⇒ p.startsWith("Char"))
// "Charizard"
```

Tip
Estos métodos NO modifican el array original (son inmutables)

Truthy y Falsy en JavaScript

Valores Falsy

Estos valores se evalúan como `false` en contextos booleanos:

`false`	`0`	`-0`
`0n`	`""`	`null`
`undefined`	`NaN`	`document.all`

Valores Truthy

Todo lo demás se evalúa como `true`:

- ✓ Objetos (incluso vacíos)
- ✓ Arrays (incluso vacíos)
- ✓ Strings no vacíos
- ✓ Números (excepto 0)
- ✓ Infinity, -Infinity
- ✓ Funciones

Ejemplos Prácticos con Pokémon

Ejemplo 1: String no vacío

```
const pokemon = "Pikachu";
if (pokemon) {
  console.log("¡Pokemon existe!"); // Se ejecuta
}
```

Ejemplo 2: String vacío

```
const pokemon = "";
if (pokemon) {
  console.log("¡Pokemon existe!"); // NO se ejecuta
}
```

Ejemplo 3: Objeto

```
const pokemon = { nombre: "Charizard" };
if (pokemon) {
  console.log("¡Objeto válido!"); // Se ejecuta
}
```

Diferencia con Java

Java (Estricto)
Solo `boolean` en condiciones

JavaScript (Flexible)
Cualquier tipo en condiciones

Actividad Práctica: Mis Primeros Pasos en JS

Instrucciones

1. Creá un archivo `mis_pokemones.js`
2. Declará variables con `let` y `const`
3. Usá template literals para crear mensajes
4. Creá funciones tradicionales y arrow functions
5. Trabajá con objetos y arrays de Pokémon

Código de Inicio

```
// Variables y Template Literals
const nombrePokemon = "Pikachu";
const tipo = "Eléctrico";
let nivel = 25;

// Función tradicional
function describirPokemon(nombre, tipo, nivel) {
  // COMPLETAR: usar template literal
  // Retornar: "Pikachu es de tipo Eléctrico y nivel 25"
}
```

Completá el Código

```
// Arrow function
const esPokemonFuerte = (nivel) ⇒ {
  // COMPLETAR: retornar true si nivel > 30
};

// Objeto Pokémon
const pokemon = {
  nombre: "Charizard",
  tipo: "Fuego",
  ataques: ["Lanzallamas", "Vuelo"],
  // COMPLETAR: agregar método ataquePrincipal()
};
```

Reto Extra

- Creá un array de 3 objetos Pokémon
- Usá `.filter()` para obtener Pokémon de tipo Fuego
- Usá `.map()` para crear array con solo nombres

Recursos
MDN Web Docs: developer.mozilla.org/es/docs/Web/JavaScript

Criterios de Evaluación

Criterio	Excelente (5)	Bueno (4)	Regular (3)	Insuficiente (1-2)
Variables y Scope	Usa correctamente let/const y comprende scope	Usa let/const pero con algún error de scope	Usa var o tiene errores frecuentes de scope	No comprende conceptos básicos de variables
Template Literals	Usa backticks y ${} correctamente	Usa template literals pero con errores menores	Mezcla concatenación y template literals	Solo usa concatenación tradicional
Funciones y Arrow	Implementa ambas sintaxis correctamente	Usa arrow functions con algún error sintáctico	Solo usa funciones tradicionales	No comprende diferencias entre ambas
Objetos y Arrays	Crea objetos/arrays y usa métodos correctamente	Crea objetos/arrays pero con errores en métodos	Crea objetos/arrays básicos sin métodos	No diferencia objetos de arrays
Truthy y Falsy	Identifica correctamente todos los valores	Identifica la mayoría pero omite algunos	Conoce los más comunes (0, "", null)	No comprende el concepto

Nota importante
La evaluación será práctica, mediante código funcional que demuestre comprensión de los conceptos.

Unidad 1 ⭐ CLAVE

DOM

Document Object Model

La unidad más importante

El DOM es la interfaz que permite a JavaScript interactuar con los elementos HTML de una página web.

Manipulación Dinámica

Cambiar contenido, estilos y estructura sin recargar la página

Creación de Elementos

Generar contenido HTML desde JavaScript

Selección de Elementos

Encontrar elementos específicos del documento

Ejemplo con Pokémon

```
// Seleccionar un elemento
const contenedor = document.querySelector('#pokemon-
container');

// Crear un nuevo elemento
const card = document.createElement('div');
card.classList.add('pokemon-card');
card.innerHTML = `<h3>Pikachu</h3>`;

// Agregar al DOM
contenedor.appendChild(card);
```

Concepto clave

El DOM es la representación en memoria del documento HTML que JavaScript puede leer y modificar.

Unidad 1 ⭐ CLAVE

Selectores del DOM

Métodos Tradicionales

getElementById

Selecciona un elemento por su ID único

```
document.getElementById('pikachu')
```

getElementsByClassName

Selecciona elementos por clase (devuelve colección)

```
document.getElementsByClassName('pokemon-type')
```

getElementsByTagName

Selecciona elementos por etiqueta (devuelve colección)

```
document.getElementsByTagName('div')
```

Métodos Modernos (Recomendados)

querySelector

Selecciona el primer elemento que coincide con el selector CSS

```
document.querySelector('.pokemon-card') // Primer .pokemon-card
```

querySelectorAll

Selecciona todos los elementos que coinciden (devuelve NodeList)

```
document.querySelectorAll('.pokemon-type') // Todos los .pokemon-type
```

Comparación Rápida

Tradicionales	Modernos
Sintaxis específica por tipo	Sintaxis uniforme (CSS)
Retornan HTMLCollection	Retornan NodeList
No aceptan selectores CSS	Más flexibles y potentes

Unidad 1 ⭐ CLAVE

innerHTML vs textContent

innerHTML

¿Qué hace?

Obtiene o establece el contenido HTML de un elemento

Ejemplo con Pokémon

```
// Agregar HTML con etiquetas
element.innerHTML = '<h3>Pikachu</h3>';
```

Precaución

No uses innerHTML con datos no confiables. Riesgo de ataques XSS.

textContent

¿Qué hace?

Obtiene o establece el contenido de texto puro de un elemento

Ejemplo con Pokémon

```
// Agregar solo texto
element.textContent = 'Pikachu';
```

Seguridad

textContent es más seguro, no interpreta HTML

¿Cuándo usar cada uno?

Usa `textContent` para texto simple y `innerHTML` solo cuando necesites insertar HTML seguro (como plantillas predefinidas).

Unidad 1 ⭐ CLAVE

createElement y appendChild

createElement

¿Qué hace?

Crea un nuevo elemento HTML del tipo especificado

Ejemplo con Pokémon

```
// Crear un elemento div
const card = document.createElement('div');

// Crear un elemento h3
const titulo = document.createElement('h3');
```

Métodos para agregar

appendChild()

Agrega un nodo al final de los hijos

append()

Agrega nodos o texto al final

prepend()

Agrega nodos al inicio de los hijos

Ejemplo Completo: Card de Pokémon

```
// 1. Crear elementos
const card = document.createElement('div');
const img = document.createElement('img');
const nombre = document.createElement('h3');

// 2. Configurar contenido
img.src = 'pikachu.png';
nombre.textContent = 'Pikachu';

// 3. Agregar estilos
card.classList.add('pokemon-card');

// 4. Construir estructura
card.appendChild(img);
card.appendChild(nombre);

// 5. Agregar al DOM
document.querySelector('#contenedor').appendChild(card);
```

Mejor práctica

Crear y configurar elementos antes de agregarlos al DOM para evitar reflows innecesarios.

Unidad 1 ⭐ CLAVE

classList - Manipulando Clases CSS

Métodos de classList

add()

Agrega una o más clases al elemento

```
element.classList.add('pokemon-destacado');
```

remove()

Elimina una o más clases del elemento

```
element.classList.remove('pokemon-oculto');
```

toggle()

Agrega la clase si no existe, la elimina si existe

```
element.classList.toggle('pokemon-favorito');
```

contains()

Verificar si una clase existe

Retorna true si el elemento tiene la clase, false si no

```
if (element.classList.contains('pokemon-legendario')) {
  console.log('¡Es legendario!');
}
```

Ejemplo Práctico

```
// Agregar clase al hacer clic
const boton = document.querySelector('#btn-favorito');
boton.addEventListener('click', function() {
  card.classList.toggle('favorito');
  if (card.classList.contains('favorito')) {
    boton.textContent = '★ Favorito';
  } else {
    boton.textContent = '☆ Favorito';
  }
});
```

¿Por qué classList y no className?
classList es más seguro y fácil de usar. Con className debes manejar cadenas completas, lo que puede causar errores.

Unidad 1 ⭐ CLAVE

Ejemplo Práctico: Cards de Pokémon con DOM

HTML 1. HTML Inicial

```html
<div id="contenedor-pokemon"></div>
```

<> 2. JavaScript Completo

```javascript
// Datos del Pokémon
const pikachu = {
  nombre: 'Pikachu',
  tipo: 'Eléctrico',
  imagen: 'pikachu.png',
  habilidades: ['Impactrueno', 'Rayo']
};

// Crear card con DOM
const card = document.createElement('div');
card.classList.add('pokemon-card');

const img = document.createElement('img');
img.src = pikachu.imagen;

const nombre = document.createElement('h3');
nombre.textContent = pikachu.nombre;

const tipo = document.createElement('span');
tipo.textContent = pikachu.tipo;
tipo.classList.add('tipo-electrico');

const habilidades = document.createElement('ul');
pikachu.habilidades.forEach(hab => {
  const li = document.createElement('li');
  li.textContent = hab;
  habilidades.appendChild(li);
});

// Ensamblar la card
card.appendChild(img);
card.appendChild(nombre);
card.appendChild(tipo);
card.appendChild(habilidades);

// Agregar al DOM
document.querySelector('#contenedor-pokemon').appendChild(card);
```

Resultado Visual



Pikachu

Eléctrico

- Impactrueno
- Rayo

Flujo de trabajo
1. Crear elementos → 2. Configurar contenido → 3. Agregar estilos → 4. Ensamblar estructura → 5. Insertar en DOM

Unidad 1 ⭐ CLAVE

Actividad Práctica: Galería de Pokémon

Objetivo

Crear una galería de Pokémon con filtros por tipo usando manipulación del DOM

Instrucciones

1. Seleccionar el contenedor de la galería
2. Crear una función que genere cards de Pokémon
3. Agregar event listeners a los botones de filtro
4. Implementar la lógica de filtrado mostrando/ocultando cards

Código de Inicio

```
// Datos de Pokémon
const pokemones = [
  { nombre: 'Pikachu', tipo: 'eléctrico', imagen: 'pikachu.png' },
  { nombre: 'Charizard', tipo: 'fuego', imagen: 'charizard.png' },
  { nombre: 'Bulbasaur', tipo: 'planta', imagen: 'bulbasaur.png' }
];

// Seleccionar contenedor
const galeria = document.querySelector('#galeria');
```

Completa el Código

```
function crearCard(pokemon) {
  // Crear card
  // COMPLETAR: crear div con classList

  // Crear elementos
  // COMPLETAR: crear img, h3, span

  // Configurar contenido
  // COMPLETAR: asignar src, textContent

  // Ensamblar card
  // COMPLETAR: usar appendChild

  return card;
}

// Agregar cards al DOM
pokemones.forEach(pokemon => {
  galeria.appendChild(crearCard(pokemon));
});

// Filtrar por tipo
const botonesFiltro = document.querySelectorAll('.filtro-
tipo');
botonesFiltro.forEach(boton => {
  boton.addEventListener('click', function() {
    // COMPLETAR: lógica de filtrado
  });
});
```

Recursos

MDN Web Docs: developer.mozilla.org/es/docs/Web/API/Document

Unidad 1 ⭐ CLAVE

Criterios de Evaluación

Excelente	Bueno	Regular	Insuficiente

Selectores del DOM

5 pts	4 pts	3 pts	1-2 pts
Usa querySelector/querySelectorAll correctamente y justifica su elección	Usa selectores adecuadamente con mínimos errores	Usa selectores pero con errores frecuentes	No usa selectores o los usa incorrectamente

innerHTML vs textContent

5 pts	4 pts	3 pts	1-2 pts
Diferencia correctamente ambos métodos y los aplica según el contexto	Comprende la diferencia pero aplica uno de forma preferente	Usa ambos métodos pero sin criterio claro	No distingue entre ambos métodos

createElement y appendChild

5 pts	4 pts	3 pts	1-2 pts
Crea y agrega elementos eficientemente al DOM	Crea elementos pero con algunos errores de estructura	Crea elementos básicos sin estructura clara	No puede crear elementos dinámicamente

classList

5 pts	4 pts	3 pts	1-2 pts
Usa todos los métodos de classList correctamente	Usa add/remove correctamente pero con dificultades en toggle/contains	Usa solo add/remove de forma básica	Manipula className directamente en vez de classList